Algorithm Benchmarking and Outcomes

The Society Connectedness & Health Lab
Yale School of Public Health

Church Closures Dashboard Project

SOCAH
SOCIETY CONNECTEDNESS AND HEALTH LAB

Table of contents

Version: 006.07.15.26
Generated using Quarto (v. 1.8.27)
and RStudio (v. 2026.01.1+403)

Preparation of the 2023 Format Raw Data

Prepared By: Shelby Golden, M.S. from the Yale School of Public Health Data Science and Data Equity team

Date Created: October 2025

Date Updated: July 29th, 2026

Overview

The project objectives were as follows:

1. **REVIEW:** Evaluate the raw data and previously outlined cleaning, validation, and harmonization methods employed by Dr. Insang Song for accuracy and identify opportunities for improvement.
2. **PREPARATION:** Implement identified improvements to prepare the dataset for analysis and distribution to qualifying collaborators.
3. **METRICS:** Calculate the metrics necessary for dynamic visualization.

Additionally, the project Principal Investigator (PI), Professor Yusuf Ransome, requested the pipeline be reproducible for future collaborators and readily adaptable to potential changes in project scope.

Following receipt of the raw data in Summer 2025, a review was conducted and findings were compared against the original methodology provided by Dr. Song. When an updated dataset was subsequently delivered in May 2026, the pipeline was expanded to accommodate two designated variations: the **2023 Format** and the **2026 Format**.

This document provides an overview of the methodological changes implemented in response to the 2023 Format "Review" report findings. The first drafts of these algorithms are described alongside an assessment of their performance and areas for improvement. General lessons learned throughout this process, such as strategies for optimizing algorithm speed and accuracy, are also discussed.

Results generated from this pipeline were used for the Summer 2025 dashboard prototype

developed by Gordon Tu.

Important Considerations

Selected code excerpts are referenced below. All snippets are drawn from scripts saved to the "Code/" directory, each prefixed by a descriptor indicating the step to which they apply:

Clean Raw Data_Step *: A discrete step in the methods pipeline outlined below. Additional classifiers may indicate whether the script was designed for use on the High Performance Computer (HPC) and/or whether it applies to the 2023 or 2026 version of the raw data.

Archival of Original Scripts

Given the time constraints of the Summer 2025 prototype launch, algorithm performance and accuracy were not improved in this iteration, and not all 2023 Formatted data could be processed — either due to algorithm limitations or time constraints.

Following the prototype release, the original scripts reflecting the step-wise methods were formalized and published on GitHub in preparation for processing the remaining 2023 Formatted data. However, upon receipt of the 2026 format in Spring 2026, much of the pipeline was overhauled to accommodate this new version, and updating the original scripts was not prioritized.

Any scripts from this pipeline that have been made available are retained for posterity only. Users should refer to the 2026 versions as the most accurate and reproducible implementation of this process.

Those seeking to understand the format-specific changes made to these methods for the 2026 Format should refer to the corresponding "Review" and "Preparation" reports.

Ignored Files

Individual-level files are withheld from GitHub and Git tracking in accordance with the Data Use Agreement (DUA) with the data provider. Many of these are stored in `**/KEEP LOCAL/`

directories or are explicitly listed under the “Project-Specific Exclusions” section of the `.gitignore` file.

Most files are available on the SOCAH Lab OneDrive for the project, while others are user-specific and do not need to be shared. If there are local-only files for this project, they will be added to the “Church-Closures-Dashboard GitHub LOCAL ONLY Files” directory. This directory will contain `README.md` files specifying the intended use and filepath for each file.

If you do not have access to the SOCAH Lab OneDrive, carefully follow the codebase instructions to establish the repository and replicate results on your own device. Raw data access instructions may vary depending on your institution.

Environment Reproducibility

Results were produced using R (v4.5.2), RStudio (v2026.01.1+403), and Quarto (v1.8.27). The `renv` package (v1.1.4) is included to reproduce the coding environment, capturing all relevant packages and their versions. If issues arise when running the scripts, verify that the environment is initialized and that the correct versions of R, RStudio, and Quarto are being used.

> **⚠ Compatibility**
>
> Updating packages, R, or Quarto may introduce compatibility issues with legacy code that will need to be resolved on a case-by-case basis.

First-time users should initialize the environment to install all required packages and ensure a reproducible coding environment. To do so:

1. In the command-line application (i.e., Terminal for Macs or Git Bash for Windows) navigate to the codebase file location.

Command-Line Interface

```
cd "FILE-PATH/Church-Closures-Dashboard/"
```

2. Launch the project by opening the `*.Rproj` file in RStudio.

3. In the R console, activate the environment by running the following lines of code:

Command-Line Interface

```
renv::restore()   # Download packages and their version saved in the
↪  lockfile.
```

> **Note**
>
> If you are asked to update packages, say no.`renv` is intended to recreate the same environment under which the project was created, making it reproducible. You are ready to proceed when running `renv::restore()` gives the output:
>
> **RStudio Console**
>
> ```
> - The library is already synchronized with the lockfile.
> ```
>
> If you experience any trouble with this step, you might want to confirm that you are using R (v4.5.2) in the RStudio (v2026.01.1+403) and Quarto (v1.8.27). You can also read more about `renv` in their vignette [1].

Methods

Approach and Methodological Objectives

The following pipeline was designed to address two primary goals:

1. Produce a cleaned and validated form of the raw data suitable for dynamic visualization at the finest regional resolution: block groups.

2. Associate each address with census boundaries defined during the 2000, 2010, and 2020 decennial years, including block group, tract, county, state, and ZIP Code Tabulation Areas (ZCTAs).

Documentation on the expected accuracy of geocoordinates provided in the raw data was not made available by the data distributor. It was therefore considered important to compare the raw geolocations against a trusted database in order to associate verified geolocations and assess the extent of error in the raw form. While not the most accurate approach, address-based database matching is a generally accepted method for confirming approximate geocoordinates [2].

Each address's geolocation coordiantes can be used to annotate each entry with the respective decennial years census boundary via point-in-polygon geocoding. The U.S. Census Bureau TIGER/Line shapefiles containing the required census boundary information provide corresponding files for decennial years requested: 2000, 2010, and 2020. However, this technique can be computationally costly; resulting in maxing in memory limits or causing preclusively slow loading if not handled strategically.

During the raw data review, it was noted that typographical errors resulted in duplicate address entries, where the associated open/closure years remained unique. Ideally, these errors would be cleanly reconciled to produce a concise dataset free of unnecessary redundancies. However, the presence of address variants is not precluding to the overall goals of this project. Mitigating these variations is nonetheless worthwhile to minimize unnecessary computational costs during geocoding and subsequent metric summarization across all addresses. Therefore, an attempt will be made to responsibly consolidate addresses using string similarity matching. Additional resolution is expected when address validation is done against a trusted database.

Special Considerations for PO Boxes

The raw data review revealed several notable special cases, detailed in subsequent sections. One of particular importance to downstream analysis involves addresses that at some point filed under a PO Box.

Later in the analysis, religious organizations are classified as closed if they did not file for four consecutive years during the observed period; organizations that subsequently resumed filing are classified as reopened. Closures are further differentiated into two categories: those that stopped operations entirely and closures due to relocation. Relocated organizations remained operational but moved outside the community, thereby resulting in a loss of social benefits to that community.

During the raw data review, two problems were identified:

a) The method employed by Dr. Insong removed all PO Box entries without removing the other addresses associated with the same business. Because the reported dates were unique across entries, this was expected to skew results, affecting approximately 12% of businesses.

b) Geocoordinates listed for PO Boxes sometimes corresponded exactly with an associated physical address. Because the validity of this association cannot be confirmed, it introduces a potential source of bias that may result in the misclassification of organizations within the "closure due to relocation" category.

For these reasons, verifying the geolocation of a PO Box is treated as zero-sum, whereas the inability to verify geocoordinates for a physical address is not considered critical. Under the following three conditions, all entries associated with a business are removed:

- Geocoordinates are absent and could not be recovered through address-based validation, preventing spatial joining.
- `address_line_1` is missing, preventing verification that the geocoordinates correspond to the institution itself rather than a general location within that ZIP code.
- PO Box entries could not be assigned valid geocoordinates through address-based geocoding.

As a final step, following cleaning and validation and prior to metric calculation, PO Box entries with verified geocoordinates will be matched to a qualifying physical address. The critical

assumption is that if a physical address exists within a nearby radius, the institution can reasonably be assumed to be operating at that location.

If no qualifying match is found, or the PO Box is too far from any nearby address, all entries associated with that business will be removed. Matching will also account for both spatial and temporal proximity; physical addresses that fall within range but are separated by intervening entries located far from the PO Box may not qualify.

A Step-Wise Framework

To achieve these goals, sequential steps were outlined, each building on the results of the preceding step.

Step 1: Address Deduplication via String Similarity Matching

> **Computational Dependencies and Custom Functions**
>
> All packages required for this step are documented in the **SET UP THE ENVIRONMENT** section of "Clean Raw Data_Step 1_2023 Format.R". Custom functions used throughout the script are in "Support Functions/For Step 1_2023 Format.R".

Different uniquely listed address line 1 (`address_line_1`) strings for a given business (`abi`) were fuzzy matched using the Jaro-Winkler method available through `stringdist(method = "jw")` in the custom function `find_similar_addresses()` [3]. This edit distance method measures string similarity by accounting for the proportion of characters shared between two strings and the number of transpositions needed to place matching characters in the same order, normalized by string length [4–6]. Scores range from zero to one, where a score of zero indicates identical strings and smaller values reflect greater similarity.

Prior to processing, addresses were normalized in the custom function `preprocess_address()` by converting all characters to lowercase, normalizing spacing and punctuation, removing non-alphanumeric characters (with the exception of commas and spaces), and trimming extraneous whitespace.

${}_nC_2$ pairwise comparisons between n unique normalized `address_line_1` strings produce a symmetric $n \times n$ string distance matrix, that is $d(i,j) = d(j,i)$ and the diagonal entries

$d(i, i) = 0$ represent the trivial self-similarity solution. To reduce redundancy, the algorithm evaluates only one orientation of each pair and omits self-similarity comparisons, computing only the upper triangle of the matrix. Addresses with a pairwise Jaro-Winkler score below a set threshold, δ, are saved as a qualifying match. The matching threshold $\delta = 0.15$ was determined empirically through testing on a sample of addresses, confirming sufficiently accurate grouping of similar entries without the erroneous inclusion of dissimilar ones.

Computational Performance

The time complexity of this operation is $\mathcal{O}\left(n^2 \times l\right)$, where n is the number of unique addresses and l denotes the computational cost of the Jaro-Winkler algorithm for a string of length l [6–8].

Because the algorithm operates solely on `address_line_1`entries, string lengths remain relatively short, making the contribution of l negligible. Additionally, because the number of unique addresses is expected to be bounded by the number of dates in the observed period, the computational demand of this method is not expected to be a limiting factor in the algorithm.

Consider the following example of a business that filed under two distinct addresses, each of which was reported with slight variations, resulting in five unique address strings.

Node	Address
1	`123 Main Street`
2	`123 Main St`
3	`123 Main St.`
4	`456 Oak Avenue`
5	`456 Oak Ave`

Using `stringdist(method = "jw")` the pairwise distances are computed for all ${}_5C_2 = 10$ unique address pairs. This yields the following upper-triangular distance matrix:

	1	2	3	4	5
1	—	0.08	0.09	0.41	0.38
2		—	0.05	0.39	0.35
3			—	0.40	0.36
4				—	0.11
5					—

Version: 006.07.15.26
Generated using Quarto (v. 1.8.27)
and RStudio (v. 2026.01.1+403)

Pairs falling below the threshold $d(i, j) < \delta = 0.15$ are considered candidate matches and recorded as edges in an undirected graph, where each node represents a unique address. This graph is stored as an adjacency list, like the one shown below [9]:

Node	Address	Nearest Neighbors	JW Distances
1	`123 Main Street`	$\{2, 3\}$	$\{0.08, 0.09\}$
2	`123 Main St`	$\{1, 3\}$	$\{0.08, 0.05\}$
3	`123 Main St.`	$\{1, 2\}$	$\{0.09, 0.05\}$
4	`456 Oak Avenue`	$\{5\}$	$\{0.11\}$
5	`456 Oak Ave`	$\{4\}$	$\{0.11\}$

A Depth-First Search (DFS) algorithm in the custom function `find_components()` is applied to the adjacency list to identify connected components, where each component forms a cluster of similar addresses.

Algorithm 1 Depth-First Search — Connected Component Identification

Require: Adjacency list `address_graph`, node set $\{1, \ldots, n\}$

Initialize an empty list of clusters
for each node i in $\{1, \ldots, n\}$ **do**
    Begin a new traversal from node i
    Initialize an empty cluster
    repeat
        Add the current node to the current cluster
        if the current node has an unvisited neighbor **then**
            Move to an unvisited neighbor and continue
        else
            Backtrack to the previous node
        end if
    until no unvisited neighbors can be reached in the current traversal
    Record the cluster
end for
Remove duplicate clusters
return list of unique clusters

One candidate address was randomly selected from each resulting cluster and assigned as the match for all other addresses within that cluster. Because the efficacy of string-based matching is expected to vary with string length, an additional geographic validation step was applied. For each set of matched addresses, the absolute difference between the maximum and minimum

reported longitude and latitude values was calculated.

For reference, one degree of longitude is approximately 69 miles (111 kilometers), while one degree of latitude varies with proximity to the equator, averaging approximately 54 miles (87 kilometers) across the contiguous United States [10,11]. The length of a typical U.S. city block similarly varies, with common estimates ranging from 100 to 200 meters [12,13]. Based on these benchmarks, address sets in which both the longitudinal and latitudinal differences exceeded 0.002 degrees (approximately 222 meters or 728 feet) were flagged as invalid matches and retained as distinct addresses in the final dataset.

This approach assumes that reported geolocations for each address are accurate to within the specified range with sufficient consistency to be reliable. As the primary objective of this method is to reduce address reduplication and improve downstream algorithmic performance, variations that may undermine this validation step are not considered to meaningfully impact the final results. When uncertain, the method is designed to err on the side of caution by retaining addresses as distinct rather than risk consolidating genuinely separate addresses that share strong surface-level similarities.

Following the algorithm outlined in **PART B: Standardize and Merge Similar Addresses** of "Clean Raw Data_Step 1_2023 Format.R", the processed ABI records identified as containing possible address reduplications were reduced to one-third of their original count. To confirm the consistency of these results, the custom function `check_all_counts_0_or_1()` was run to assess whether duplications had been introduced in the year-opened and year-closed columns, which had been confirmed as unique in the raw data. Unfortunately, 2.6% of ABI records were flagged as having newly introduced duplications.

As the algorithm was developed under time constraints, its deduplication strategy could not be optimized. Priority was instead placed on progressing through subsequent steps. Consequently, records with induced duplications were excluded from the prototype pipeline, while the nature of the duplications was later carefully assessed to inform a strategy for their reintegration and to guide improvements to the overall approach. These insights informed the design of the 2026 Format comparative version.

Additional details regarding the algorithm's performance outcomes can be found in Results - Step 1: Address Deduplication via String Similarity Matching.

Step 2: USPS API Address Validation

> **Computational Dependencies and Custom Functions**
>
> All packages required for this step are documented in the **SET UP THE ENVIRONMENT** section of "Clean Raw Data_Step 2_2023 Format.R". Custom functions used throughout the script are in "Support Functions/For Step 2_2023 Format.R".
>
> The algorithm implementing the described methods has been separated into a dedicated HPC script, contained in "Clean Raw Data_Step 2 HPC_2023 Format.R", which includes its own **SET UP THE ENVIRONMENT** section. **SUBSECTION A1: Utilizing the HPC** further describes how to reproducibly run the script within the HPC environment as both a live session or batch array. The preconfigured shell script used for deploying batch arrays is "USPS SLURM_2023 Format.sh".

As described in the Results - Step 1: Address Deduplication via String Similarity Matching section, the deduplication step reduced the number of entries associated with ABIs with multiple listed addresses to approximately 36.4% of the original count. However, the step had the unintended effect of inducing duplications in the columns containing the years-open and years-closed binary indicators for 2.6% of ABIs.

This substantial reduction indicates a significant degree of address variation within the dataset. However, excessive deviation from standard address formatting has the potential to contribute to failed matches with the U.S. Census Bureau's Geocoder API, used in **Step 3** to validate the geocoordinates associated with each address [14]. To mitigate this risk, addresses were first validated against the USPS Address API to retrieve the preferred, standardized form of each address prior to geocoding [15].

> **Alternative Validation Resources**
>
> These APIs were chosen primarily for their relative simplicity of setup and use, their suitability for automation and parallelization within the HPC environment, and their lack of associated cost. However, as described in their respective results sections, these processes were unable to validate as many records as anticipated. Furthermore, as they are separate systems, they required sequential processing, whereas alternative resources can offer both address and geolocation validation within a single query. The following alternative databases were considered for validating the listed physical

addresses and associated geolocations:

- Environmental Systems Research Institute, Inc. (Esri) StreetMap Premium, including the locally hosted version available to Yale affiliates
- Google Maps API

These sources were ultimately not used due to a combination of limiting factors. Under the applicable Data Use Agreements (DUAs), it was necessary to confirm that no API usage would violate the terms of those agreements. The USPS and U.S. Census Bureau APIs were approved following coordination with the relevant security review processes; however, accessing the Esri online database or Google Maps API required additional review.

The review process was initiated for the Google Maps API given its anticipated capacity to efficiently validate addresses and geolocation. However, the review extended beyond the project timeline, rendering those resources inaccessible. Additionally, Esri requires manual adjudication of alternative address suggestions, which would have been prohibitively time-intensive. Researchers from the Yale Center for Geospatial Solutions (YCGS) further noted that the online Esri resource is more current and robust than its locally hosted counterpart, suggesting that the local version would offer little advantage over the USPS Address database in terms of address validation quality.

The algorithm queries the USPS Address API using the API specifications, methods, and sample processes outlined in the official developer documentation and the USPS API Examples GitHub repository [15,16]. Pre-configured customer key and secret credentials are used to generate an API token for each address query via the custom function `generate_usps_token()`. The raw address fields, address lines 1 and 2, city, state, and five-digit ZIP code, are submitted to the database, and the response JSON is parsed to extract validated address components into discrete columns via the custom function `validate_usps_address()`. Both functions utilize the `httr` and `jsonlite` packages for API interaction [17,18].

If the initial USPS validation failed to return a match, the script employed two fallback strategies: city and ZIP code correction, and using address lines 1 as 2 instead. If neither strategy produced a match, the query returned “No address match found”.

The first alternative strategy was of particular importance for this dataset. During the initial raw data assessment, it was revealed that the ZIP code columns had at some point been converted to numerical values, causing leading and trailing zeros to be stripped. Prior to querying, these

ZIP codes were normalized to the expected five-digit length; however, padding zeros were added arbitrarily.

To mitigate potential match failures against the API database, candidate ZIP code variations were generated using the custom function `make_zip5_candidates()`. For example, `make_zip5_candidates("01230")` would generate "01230", "00123", and "12300". The custom function `get_city_info()` then retrieved the recommended city associated with each candidate ZIP code. Valid ZIP code and city pairings were subsequently used in place of the raw values to resubmit the query, though the algorithm only attempted the first valid pairing returned by `get_city_info()`. The look-up table was prepared from the SimpleMaps Basic United States Cities Database (version 1.90) using the custom function `build_zip_city_lookup()` [19].

> **API Credentials and Configuration**
>
> Users who wish to utilize this resource for their own work will need to register for a USPS Developer account, sign the license agreement, and configure their own individual API client key and secret.
>
> The client key and secret must be kept private and must not be shared. Instructions for configuring these credentials are provided in the header of "Code/Clean Raw Data_Step 2_2023 Format.R".

> **USPS API Version 3.3.1 Update: Compatibility and Pricing**
>
> As of August 1st, 2026, the USPS upgraded the API from version 3.2.3 to 3.3.1 and updated their terms of use to include tiered pricing for API access.
>
> Users should be aware that the algorithm reported here was designed for version 3.2.3 and may not be fully compatible with the updated API without modification. Additionally, the 2023 Format implementation assumes unrestricted access to the API, whereas the 2026 Format implementation was updated with a version 2 allowing users to specify the number of addresses queried against the USPS API, enabling cost management.

This process was designed to be run both locally and within Yale's HPC environment. Within the HPC, users additionally have the option to process results either in a live session or through a batch array.

After preparing the required subset in **SUBSECTION A1: Prepare Subset for HPC** of "Clean

Raw Data_Step 2_2023 Format.R", users run the algorithm outlined in "Clean Raw Data_Step 2 HPC_2023 Format.R". **PART A: UTILIZING THE HPC** details the required environment configuration, necessary file uploads, and instructions for running the script within the HPC via either a live session or batch array. The file "USPS SLURM_2023 Format.sh" is the preconfigured shell script used to execute the batch array.

Following this run, the custom function `qc_validation_results()` was used to validate the results and ensure consistency with expectations across all data chunks prior to merging. The function performs six diagnostic checks: verifying the dimensions of each chunk, assessing total row coverage, validating that each row and file falls within its expected row range, and summarizing the distribution of address verification outcomes per file.

To address duplications introduced during **Step 1**, additional collapsing was applied to the 10% following address verification. Exact duplicates where year-opened and year-closed values are identical were resolved by retaining a single record at random with `distinct()`. Entries not resolved by this approach were repaired by restoring raw data values.

Repaired entries were quality-checked for dataset mismatches and missing year-opened or year-closed values. The case was rare, likely reflecting cases where verified addresses or cluster assignments no longer corresponded to raw data entries, and were excluded. Together, these steps resolve all residual duplications in the dataset.

Additional details regarding the algorithm's performance outcomes can be found in Results - Step 2: USPS API Address Validation.

Step 3: Address Geocoding

> Computational Dependencies and Custom Functions
>
> All packages required for this step are documented in the **SET UP THE ENVIRONMENT** section of "Clean Raw Data_Step 3_2023 Format.R". Custom functions used throughout the script are in "Support Functions/For Step 3_2023 Format.R".

The dashboard visualizes aggregated addresses to show metrics summarized to different census boundary levels: block group, tract, county, state, and ZIP Code Tabulation Areas (ZCTA). For the Summer 2025 prototype, results needed to at least support aggregation to the county level. While the exact location of addresses is not critical for county-level aggregation, seemingly minor deviations can greatly influence granular census boundary results, such as

block groups and tracts.

Additionally, in Results - Step 1: Address Deduplication via String Similarity Matching it was noted that geocoordinates for addresses sharing the same `address_line_1`, and hence potentially the same physical location, varied by as much as 3 degrees (Table 1). At mid-latitudes, 3 degrees corresponds to roughly 150–175 miles, approximately the distance between New Haven, CT and Philadelphia, PA.

To mitigate these errors, addresses verified in **Step 2** are geocoded using the U.S. Census Bureau's Geocoder API to retrieve the geocoordinates assigned to that address [14]. The API's documentation does not specify which part of the building the coordinates represent, such as the delivery entrance or rooftop centroid [20]. All available address fields (address line 1, city, state, and five-digit ZIP code) were used to find matches, prioritizing verified addresses over raw, given values.

The raw address fields are submitted to the database, and the response JSON is parsed to extract validated address components into discrete columns via the custom function `validate_geocoordinates_geoCoder()`. This function utilizes the `httr` and `jsonlite` packages for API interaction [17,18]. Each query URL, shown below, is constructed using the parsed address components, the benchmark "Public AR Current", and the vintage "Census 2020". As the API query would occasionally time out, each failed query was resubmitted up to five times before moving on.

RStudio Console

```
# Construct the request URL for the Census Geocoder API.
base_url <- "https://geocoding.geo.census.gov/geocoder/geographies/address"
request_url <- paste0(
  base_url,
  ↪  "?format=json&benchmark=Public_AR_Current&vintage=Census2020_Current",
  "&street=", URLencode(street),
  "&city=", URLencode(city),
  "&state=", URLencode(state),
  "&zip=", URLencode(zip)
)
```

If the response JSON returned more than one possible match, the first result was selected without further adjudication. The returned geocoordinates were then compared against the reported geocoordinates to determine whether their difference was less than 1 degree.

Additional details regarding the algorithm's performance outcomes can be found in Results - Step 3: Address Geocoding.

Step 4: Linking PO Boxes to Physical Addresses

> **Computational Dependencies and Custom Functions**
>
> All packages required for this step are documented in the **SET UP THE ENVIRONMENT** section of "Clean Raw Data_Step 4_2023 Format.R". No custom functions were used in this script.

Communities can be equally impacted by the loss of social services or other support offered by a religious institution if it closes or relocates outside of the community. To capture this, closures resulting from a relocation are also quantified, though at varying gradients, as some attendees may be able to travel greater distances than others. This effort does not attempt to reconcile the potential role of digital or remote service delivery; instead, it focuses exclusively on the impact to physical, in-person services.

In order to support this computation, PO Boxes need to be specially handled. A possible assumption is that business owners open PO Boxes near a physical location. If this assumption holds, then the immediately preceding or following physical address should be reasonably close to the listed PO Box.

> **PO Box Geospatial Proximity Method (Prototype Only)**
>
> The Yale Center for Geospatial Solutions (YCGS) recommended associating PO Boxes with physical addresses by creating simple features for each distance boundary around a community and applying point-in-polygon matching. However, this method is time-intensive, and the project PI did not have a specific distance threshold in mind and instead wanted to explore distances empirically.
>
> The k-means clustering method described here was developed to quickly assess the relative spatial proximity of addresses and PO Boxes associated with a given business. Due to the computational cost of these assessments and project time constraints at the time of development, geographic proximities were kept broad and not all assumptions were validated prior to the prototype release.

Additionally, the assumption that PO Boxes could be reliably associated with a nearby physical address was later determined to be too strong to justify its use in the final methods. Instead, businesses with a PO Box listed within the selected date range are excluded from the metrics calculation altogether.

The relocation identification method was completely revised for the 2026 Formatted version. This document is retained for posterity and reflects the method attempted for the prototype.

Geocoordinates Preferred Over Census Boundaries for Consistency

The original methods from Dr. Song quantified relocations by address change: different census tract, city, county, or state; and by coordinate distance: $\geq$ 100 meters, $\geq$ 500 meters, or $\geq$ 1000 meters. The physical distance implied by a census boundary change varies depending on whether the region is densely populated (smaller boundaries) or rural (larger boundaries). Therefore, the focus will remain on coordinate-based distances to ensure outcomes are treated more consistently.

The `dbscan(eps = 1, minPts = 2)` function groups observations into clusters when at least two points fall within a radius of 1 (in the units of the input coordinates) [21]. Points that do not meet this density requirement are labeled as noise (0), while all other points are assigned to cluster groups 1, 2, etc. Cluster labels are applied independently within each business ID. As shown in the Results - Step 4: Linking PO Boxes to Physical Addresses, this method inconsistently clustered coordinates that would otherwise be considered a relocation of meaningful distance.

Given time constraints, the prototype was scoped to focus only on addresses not suspected to have relocated outside their county. Quantifying relocations at a finer scale was to be revisited after the symposium. Addresses were therefore first filtered to those assumed to have not moved, defined as a maximum longitude and latitude difference of less than 0.5 degrees across all addresses for a given business, then clustered into areas and aggregated prior to assigning census boundaries.

Consolidated records were then annotated using address information and validation metrics drawn from one of the aggregated values. Priority was given to entries with both a verified address and verified geocoordinates from **Step 2** and **Step 3**, with additional conditions applied to rank the remaining cases in the following order: verified address > verified geography > first row. Consolidating records into localized clusters greatly expedited the point-in-polygon spatial assignment, enabling results to be presented at the prototype symposium; however, this

was not considered a good approach for the final methods where more time was available.

Additional details regarding the algorithm's performance outcomes can be found in Results - Step 4: Linking PO Boxes to Physical Addresses.

Step 5: Point-in-Polygon Spatial Assignment

> Computational Dependencies and Custom Functions
>
> All packages required for this step are documented in the **SET UP THE ENVIRONMENT** section of "Clean Raw Data_Step 5_2023 Format.R". Custom functions used throughout the script are in "Support Functions/For Step 5_2023 Format.R".

The data spans three decennial years: 2000, 2010, and 2020. Because users can select varying date ranges and may expect temporally accurate census boundary mappings, each address must be annotated with the census boundaries associated with all three. Once annotated, entries can be aggregated to the census boundary identifier from a given decennial US Census Bureau's TIGER/Line Shapefile release [22]. This enables results to be spatially projected onto maps via the decennially accurate demarcations of census boundaries.

Accurate spatial depiction of these trends across census boundary levels requires consideration of varying degrees of locational error. Counties and states represent larger areas, making them less sensitive to imprecise locational information and less prone to boundary changes between decennial periods. In contrast, smaller boundaries such as block groups and tracts can change substantially between decennial periods and are more sensitive to locational errors. Ensuring accurate fine-level annotations for historical decennial years is therefore an important consideration.

Steps 2 and **3** seek to validate and improve the geocoordinates associated with a given business through address-based geocoding. When associating census boundary identifiers with these records, there are two fundamental approaches:

1. Referencing a database with decennial boundary information already associated with each address.

2. Assigning decennial boundaries to each address via point-in-polygon spatial joining of verified longitude and latitude coordinates against reliable spatial representations of map boundaries, known as polygons.

The second approach is the standard of practice, ensuring all desired decennial boundary information is correctly assigned to the provided geocoordinates. Its accuracy is limited only by the reliability of the available address coordinates and boundary shapefiles. Figure 1 demonstrates how geocoordinate points are interpreted by a spatial join to polygon geometries, such as county-level census boundaries, geographically.

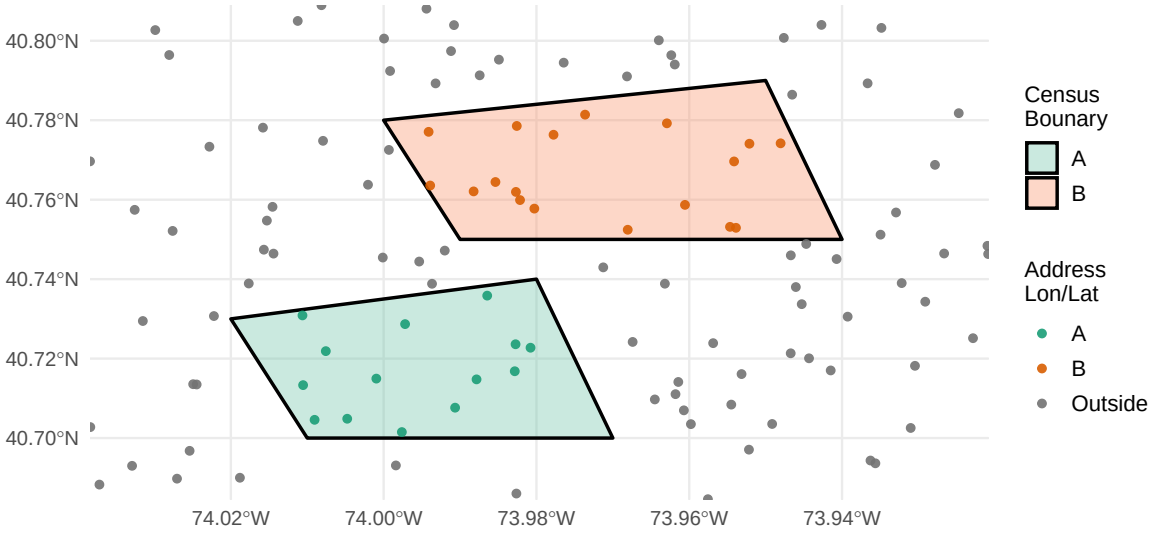

Figure 1: Illustration of Point-in-Polygon Spatial Joining

Computational Performance

The time complexity of a naïve spatial join for a single decennial period is $\mathcal{O}\left(N \times M\right)$, where N denotes the number of points and M the number of polygons [7,23].

This can create significant time lags when associating multiple addresses across many boundaries, such as block groups or tracts in a densely populated region. Steps leading up to the point-in-polygon operation reduce the number of comparisons by consolidating addresses and filtering regions down to the state level. The county and other census boundaries were not used for filtering, as they were assumed to vary between decennial periods.

Other strategies exist that can improve the speed of this operation. For example, spatial filtering or indexing can reduce the complexity to $\mathcal{O}\left(N \log(N) \times M \log(M)\right)$ [23]. Due to time constraints developing the process, these were not explored, and some entries, particularly those spanning denser census regions with more addresses, took orders of

Version: 006.07.15.26
Generated using Quarto (v. 1.8.27)
and RStudio (v. 2026.01.1+403)

> magnitude longer to compute than others.

Three sources of census boundary information were explored:

a. US Census Bureau Geocoder: Used in **Step 3** to validate geocoordinates; also provides a database of addresses already annotated with census boundary information [14,20].

b. `tidycensus`: Interfaces with select data from the US Census Bureau APIs and imports them as `tidyverse`- and `sf`-ready objects. Data accessible includes TIGER/Line Shapefiles [24].

c. `tigris`: Downloads TIGER/Line Shapefiles directly from the US Census Bureau [25].

All three ultimately draw from the US Census Bureau, but each has its own limitations. For example, the Geocoder was considered because it is already used in **Step 3**, potentially allowing both steps to be combined into a single query. Its pre-annotated addresses would also eliminate the need for point-in-polygon spatial joining, reducing both computational costs and processing time. However, it is restricted to the 2010 and 2020 decennial periods and does not provide all the required boundaries.

`tidycensus` was another viable option but also did not offer all the required census boundaries. Ultimately, `tigris` was selected for its ease of use and straightforward implementation via the custom functions `find_census_geographies_sf()` and `get_tracts_for_year_and_state()`. The `tidycensus` algorithm counterpart was not retained, but the Geocoder algorithm, implemented via `get_census_tract_geocoder()` utilizing the `httr` and `jsonlite` packages for API interaction was kept for reference [17,18].

Spatial joining was conducted using the `sf` package [26]. Geocoordinates were converted to the standard World Geodetic System 1984 (WGS84) and projected into the coordinate reference system (CRS) of the census boundary polygons. The algorithm for retrieving census boundary information via `tigris` and joining it to the data is outlined below.

Additional details regarding the algorithm's performance outcomes can be found in Results - Step 5: Point-in-Polygon Spatial Assignment.

Algorithm 2 Spatial Join to Assign Census Tracts (2000/2010/2020) via `tigris`

Require: Address table `address_data` with `lon`, `lat` in WGS84; `years` $\subseteq \{2000, 2010, 2020\}$

Ensure: For each address and year: matched `STATEFP`, `COUNTYFP`, `GEOID`, `NAMELSAD`

Drop rows with missing `lon` or `lat`
for each `year` $\in$ `years` **do**
    Load state polygons (TIGER/Line): `states_sf` $\leftarrow$ `tigris::states(year = year, class = sf)`
    Convert addresses to sf points: `pts` $\leftarrow$ `as_sf(address_data, (lon, lat), crs = 4326)`
    Transform standardized `pts` to the coordinate reference system (CRS) used by the boundary polygons, `CRS(states_sf)`
    Spatially join points $\rightarrow$ states (within): `pts_state` $\leftarrow$ `join(pts, states_sf, within)`
    Extract `state_fips` $\leftarrow$ `pts_state.STATEFP`
    for each `state` $\in$ `unique(state_fips)` **do**
        if `state` is missing **then**
            continue
        end if
        Fetch decennial-specific tract shapefiles (tigris) and standardize keys: `tracts_sf` $\leftarrow$ `get_tracts_for_year_and_state(year, state)`
        if `tracts_sf` $=$ `NULL` **then**
            continue
        end if
        Subset points in `state`: `pts_s` $\leftarrow$ {points in `pts_state` : `state_fips` $=$ `state`}
        Transform `pts_s` to `CRS(tracts_sf)`
        Spatially join points $\rightarrow$ tracts (within): `pts_tract` $\leftarrow$ `join(pts_s, tracts_sf, within)`
        Keep standardized outputs: `STATEFP`, `COUNTYFP`, `GEOID`, `NAMELSAD` and set `Year` $\leftarrow$ `year`
        Append `pts_tract` to results
    end for
end for
return combined results across all years/states

Algorithm Performance Optimization

During the initial development phase, from late July through the first prototype unveiling on June 18th, 2025, priority was placed on rapidly establishing a functional data processing workflow rather than on computational optimization or algorithmic robustness. As a result, many processing steps were slow to execute, and only a portion of the full dataset could be processed in time for the prototype presentation. Many previously undetected edge cases also failed to process correctly, requiring extensive post-processing intervention to reintegrate affected records into the final dataset.

Following the symposium, it was decided that running the pipeline on Yale Center for Research Computing (YCRC) infrastructure could substantially reduce processing time. Consultation with researchers at the YCRC identified two critical considerations prior to deployment:

1. **Script Efficiency:** Processing scripts must be optimized prior to HPC deployment, both to ensure the responsible use of shared computational resources and because HPC environments will not always be able to compensate for inefficient code.
2. **API Constraints:** Steps dependent on external API calls are bounded by those services' rate limits and response latencies. These bottlenecks are external to the pipeline and might not be resolved by parallelization.

This section reports how the processing scripts were assessed for computational efficiency. Specific optimization opportunities identified during full-dataset processing are addressed in the results section corresponding to each respective method step.

Functions with long local runtimes despite parallelization via the `future.apply` package were profiled using `profvis()`, an R tool for identifying where R spends the most time during execution. This analysis identified two operations as primary contributors to overall processing time: data subsetting and row-wise data combination (row-binding).

To address these bottlenecks, three common R approaches were benchmarked: base R, the `tidyverse`, and `data.table`. Within each approach, specific strategies were also compared. For example, `dplyr::filter()` versus `dplyr::semi_join()` for subsetting, and `data.table::rbindlist()` applied within a loop versus after loop completion for row-binding.

Overall, processing a large data frame using the `data.table` class proved to be the most computationally efficient approach (Figure 2). Additionally, applying `data.table::rbindlist()` after a loop was found to be more efficient than base R's `do.call()`, while sequentially

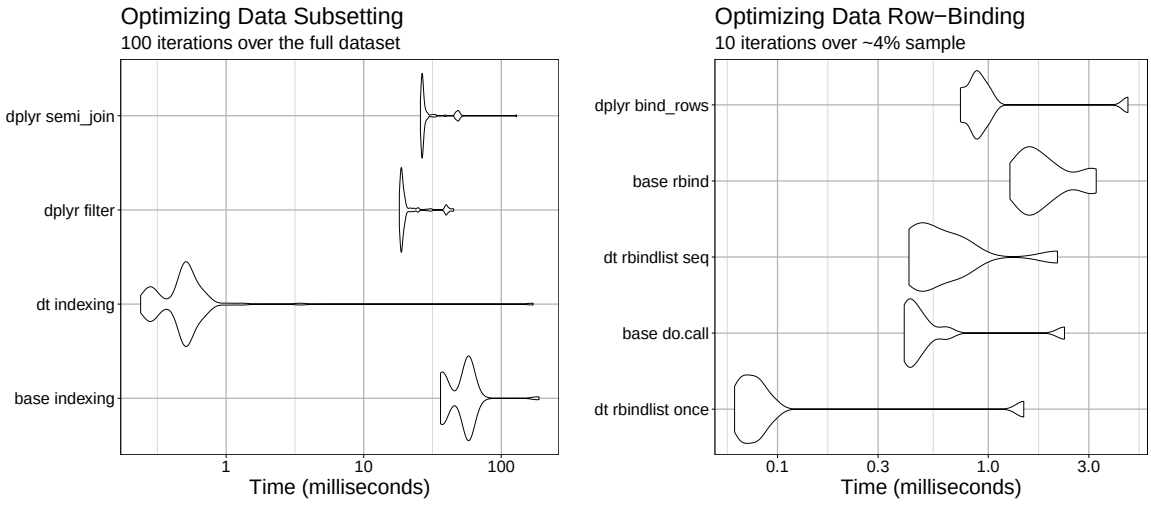

Figure 2: Microbenchmark Timings: Approaches to Subsetting and Row-Binding

applying `data.table::rbindlist()` within a loop yielded comparable results to the latter.

Refer to Appendix - Optimizing Data Subsetting and Appendix - Optimizing Data Combination for information on where to find the code that generates these results, as well as the results codebook.

Results

Step 1: Address Deduplication via String Similarity Matching

Explore the Nature of Duplications Added

The deduplication algorithm in **PART B: Standardize and Merge Similar Addresses** reduced the number of entries associated with ABIs with multiple listed addresses to 36.4% of the original count. A column-wise sum over each date of entry was conducted using the custom function `check_all_counts_0_or_1()` to confirm if all years-open and years-closed binary indicators remained unique. However, the step had the unintended effect of inducing duplications for 2.6% of ABIs.

It is possible that these duplications arose either from the same address falling into multiple clusters, or from addresses sharing an identical `address_line_1` but differing in metadata,

causing artificial expansion of the table when a randomly chosen match had multiple rows with distinct metadata. To begin investigating this, we first assess whether the induced duplicates provide identical records for the years-open and years-closed binary columns. Such cases can be trivially reconciled by randomly removing all but one instance.

The `duplicated()` function marks the first occurrence of a record as `duplicated = FALSE` and all subsequent occurrences as `duplicated = TRUE`. As `FALSE` is the default value, any set of detected duplications will always contain at least one `FALSE` entry, corresponding to the first occurrence of the record. In this assessment, the data was restricted to ABI records containing at least one duplicated years-open or years-closed entry. Duplications were assessed using the custom `check_duplicates_unique_info()` function across all years-open and years-closed columns, grouped by unique `abi` and `address_line_1`.

RStudio Console

```
## --------------------
#SUBSECTION B2: Explore the Nature of Duplications
## ~ line 507

# Initialize progress bar
pb <- txtProgressBar(min = 0, max = nrow(finish_build[finish_build$abi %in%
↪  dup_added, ]), style = 3)

new_info <- mutate_with_progress(
  # Subset the data to include only ABIs with new duplicates.
  df = finish_build[finish_build$abi %in% dup_added, ],
  # Specify the columns to check for duplications.
  cols_to_convert = grep("^20", names(finish_build), value = TRUE),
  # Define the columns to group by.
  grouping_cols = c("abi", "address_line_1"),
  # Custom function to check for duplicates and set "is_unique".
  conversion_func = check_duplicates_unique_info,
  # Progress bar to visually track the conversion process.
  pb = pb
)
```

	Duplicate = TRUE		
Duplicate = FALSE	0	1	2
0	8074	0	0
1	0	6527	44
2	612	6	30
3	0	7	0

Most entries indicate that the duplicated information is identical, suggesting that duplicates can be trivially resolved by randomly selecting one record from each set. This applies to entries with equal numbers of `FALSE` and `TRUE` results (n = 14,631) and entries with one `FALSE` and a `TRUE`-majority outcome, the latter indicating multiple duplicates associated with a single address (n = 44). These likely resulted from the matched address containing multiple metadata variations.

Among the 44 entries exhibiting more than one `TRUE` value, variation was observed in both city and zip code: three distinct cities were recorded per entry, and 97.7% of entries listed a differing zip code, while the state remained consistent across all records. Most of these entries shared an identical `address_line_1`, while a small number had no value recorded for that column.

> **Interpreting Records with Identical `address_line_1` Values**
>
> It is a plausible assumption that entries sharing an identical `address_line_1` but differing in other metadata represent the same physical address, given the low probability that a business would register an identical `address_line_1` across distinct locations. Valid city variation may also arise for addresses situated near administrative boundaries or spanning decennial census years.

As expected, table 1 shows that all entries lacking an `address_line_1` value failed the geolocation similarity test, with the minimum difference for both longitude and latitude exceeding 0.002 degrees. Notably, entries sharing an identical `address_line_1` also failed at an average rate of approximately 85%, suggesting that variation in city and zip code may produce differing reported geolocations even when the physical address is reasonably expected to be the same. Consequently, the geolocation test results for these entries will be overridden to indicate a passing match.

Table 1: Geocoordinate Summary Statistics for One `FALSE` and `TRUE`-majority Outcomes

Has an `address_line_1`

Statistic	Longitude	Latitude
Min.	0.0009	0.0060
1st Qu.	0.0306	0.0310
Median	0.1387	0.0945
Mean	0.2524	0.1855
3rd Qu.	0.1901	0.1650
Max.	3.2525	1.0836

Has NO `address_line_1`

Statistic	Longitude	Latitude
Min.	0.1492	0.0038
1st Qu.	0.2589	0.0639
Median	0.6446	0.4595
Mean	0.9893	0.4549
3rd Qu.	1.3749	0.8504
Max.	2.5188	0.8968

4% of ABI records assessed contain information that differs between the years-open and years-closed duplications, meaning these cases cannot be trivially resolved. This applies to entries containing only `FALSE` values, which exhibited differing patterns across the years-open and years-closed binary columns (n = 612). These likely resulted from inconsistent clustering, which allowed column-wise sums to differ across duplicated entries. Entries with a `FALSE`-majority outcome contain either records unaffected by the duplication error, or records in which one address has binary column values distinct from those of the remaining duplicated entries (n = 13).

A small number of representative examples from this `FALSE` only category were closely examined alongside the corresponding `find_similar_addresses()` results. These cases suggest that addresses where duplicated entries exhibit conflicting years-open and years-closed binaries likely reflect errors in associating a candidate match within its cluster. This behavior mimics the results expected from overlapping clusters generated by `find_similar_addresses()`, suggesting instead that the algorithm occasionally selected an incorrect matching address outside of the expected cluster.

Addresses that were not clustered together, or are assumed to share a physical location despite differing metadata, may resolve to the same suggested address following validation with the USPS API. Accordingly, these entries will be expanded and annotated to differentiate their respective detected error patterns, then aggregated again following validation where applicable. The same consolidation rules will apply, with the exception of entries sharing an identical `address_line_1` value where the geolocation test will be overridden.

Similar Addresses: Geolocation Discrepancies

Approximately 15% of entries failed the geolocation similarity test, and fewer than 1% failed to complete it.

All entries that failed to complete the test have `NA` values in both the longitude and latitude fields. These entries will be retained for the time being, and any that do not match the geolocation referential database will be removed. After inspecting a small number of examples, it is evident that some may represent correctly identified similar addresses but the provided longitude and latitude values fall outside the 0.002 degree error threshold. In some cases, these entries share not only the same `address_line_1` value, but the same address metadata as well.

To assess the prevalence of entries sharing an identical address, the number of clusters generated under two conditions will be compared: one using relaxed string similarity settings at the same threshold as above, $\delta = 0.15$, and one requiring an exact match, $\delta = 0$. The custom function `process_abi_group()` generates these two outcomes as a list.

RStudio Console

```
## --------------------
## SUBSECTION B3: Similar Addresses: Geolocation Discrepancies
## ~ line 862

original_addresses <- church_wide %>%
  filter(abi %in% not_same$abi) %>%
  mutate(compiled_address = paste(address_line_1, city, state, zipcode, sep =
  ↪  ", "))

# Initialize the base R text progress bar
total_abis <- original_addresses %>% distinct(abi) %>% nrow()
pb <- txtProgressBar(min = 0, max = total_abis, style = 3)

# Process each ABI group separately and update the progress bar
count_addresses <- original_addresses %>%
  group_by(abi) %>%
  group_split() %>%
  map2(1:total_abis, ~process_with_progress_txt(pb, .x, process_abi_group,
  ↪  .y)) %>%
  names<-(original_addresses %>% distinct(abi) %>% pull())

address_counts <- count_sublists(count_addresses) %>%
  mutate(abi = as.numeric(abi))
```

Almost two-thirds ($\approx$ 63%) involve addresses that are exact duplicates. For these, the geolocation test result will be overridden and marked as passed. The remaining records will be

split back into separate entries and re-evaluated following address validation. Once address validation is complete, the following handling steps will be applied:

- **All records validate to distinct addresses**: retain as separate entries.
- **Multiple records validate to the same address**: override the geolocation test result and merge into a single record using the validated address and the average of their longitude and latitude values.

Concluding Thoughts

Trivial duplications were handled prior to expanding entries for subsequent address validation. This process reduced detected duplications by approximately 60%, and the total combined dataset was reduced by approximately 53% relative to its raw form. Examination of entries with a failed geolocation test additionally revealed that the exact same address had varied geolocation points of up to 3 degrees, underscoring the importance of verifying the geolocation of all entries.

Refer to Appendix - Step 1: Address Deduplication, Step 1: Verify No Duplicates Got Added, Step 1: Override for Addresses With Same Physical Address Line 1, Step 1: Resolving Failed Geolocation Tests, and Step 1: Final Result for information on where to find the code that generates these results, as well as the results codebook.

Step 2: USPS API Address Validation

All compiled data passed the quality checks defined in `qc_validation_results()`. Notably, address verification rates varied substantially across arrays: approximately 30% of arrays validated fewer than 10 out of 42,000 addresses, the highest verification rate observed was 30%, and the average was approximately 6%. Most addresses that were verified either passed or did not require a geolocation test. Only 5% of PO Box entries were verified against the API, but most that were consolidated passed the geolocation test.

As described in the **Step 2** methods, duplicates in the year-opened and year-closed columns were intentionally retained through address verification to support directed secondary handling. Duplicates were then reconciled in two stages: trivial cases were resolved via `distinct()`, and remaining entries were repaired by patching in raw year-opened or year-closed values. Of all induced duplicates, 20% were resolved via `distinct()`, with the majority requiring raw data repair. Approximately 5% remained unresolved by either method, affecting approximately

0.09% of businesses in the dataset. Given the negligible impact, these entries were marked for removal rather than processed further.

Entries flagged with `override_duplicate == TRUE` following **Step 1**, those with an identical `address_line_1`, were matched to a verified address, prioritizing earlier-listed candidates over later ones. Approximately 0.02% of ABI records were processed in this step, of which 35% were successfully matched to a verified address. Reconciling duplicates resulted in a slight increase in entries (2.5%) from the previous step.

Refer to Appendix - Step 1: Address Deduplication, Step 1: Verify No Duplicates Got Added, Step 1: Override for Addresses With Same Physical Address Line 1, Step 1: Resolving Failed Geolocation Tests, and Step 1: Final Result for information on where to find the code that generates these results, as well as the results codebook.

Step 3: Address Geocoding

Due to time constraints, only one-third of the data processed through **Step 2** could be evaluated in this step prior to **Step 4**. These records were randomly selected to broadly represent counties across the contiguous US.

81% of these were successfully geocoded, 16% returned no match for the provided address, and the remainder failed to complete the request. All verified geocoordinates fell within 1 degree of the provided geocoordinates. A higher proportion of addresses matched by the geocoding database were verified (86%), compared to 46% of unverified addresses. Additionally, unverified addresses had a higher rate of API failures, likely due to query timeouts, at 10% compared to 1% of verified addresses.

	Verified Geocoordinates		
Verified Address	**API failed**	**No match**	**TRUE**
TRUE	0.98	12.35	86.67
FALSE	10.34	43.60	46.06

As noted in Results - Step 2: USPS API Address Validation only 5% of PO Box entries were verified by the USPS API overall. Within the processed subset, this proportion enriched to 95%. Despite this, none of the verified PO Boxes were matched in the geocoder database.

	Verified Geocoordinates		
PO Box	**API failed**	**No match**	**TRUE**
FALSE	2.25	10.21	87.54
TRUE	0.00	100.00	0.00

Step 4: Linking PO Boxes to Physical Addresses

Development of this step and processing data through it were impeded by the short timeline leading up to the Summer 2025 symposium. In Methods - Step 4: Linking PO Boxes to Physical Addresses, many of these design limitations and time-driven choices are described. As a further consequence of the time constraints, only half of the data processed through **Step 3** could be evaluated through this step prior to **Step 5**.

Gross location changes are classified by the maximum difference in longitude and latitude coordinates listed across all addresses for a given business. To get a sense of maximal location change, different distance bands are approximated. Note that the relationship between degrees and Euclidean distance differs between longitude and latitude and varies by location on the Earth's surface. This check intentionally simplifies those details to provide a quick, broad, high-level sense of address movement for a given business in the contiguous US. For reference, 0.002 degrees $\approxeq$ 222 m, or approximately two city blocks, translating to approximately 111,000 m per degree.

dist_band	n	pct
within 1 block	264300	85.5
within 1 mile	15334	4.96
1–5 miles	22182	7.18
5–10 miles	4937	1.6
10–50 miles	2050	0.66
$>$ 50 miles	245	0.08

Nearly 86% of businesses remained within one block of each other. An additional 12% had a relocation within five miles, 1.6% between five and ten miles, and less than 1% moved more than ten miles, of which 0.08% moved more than 50 miles.

Clustering Validity Assessment for Detecting Business Address Changes

Table 2 parses clustering results by number of unique addresses associated with a business and by maximum longitude/latitude coordinate difference detected.

Table 2: Row Count and Cluster Composition (by # Rows and by Distance Band)

By # Rows & # Clusters

	# Clusters			
# Rows	**0**	**0, 1**	**1**	**1, 2**
2	146	0	38,440	0
3	0	43	5,252	0
4	0	7	722	2
5	0	0	109	0
6	0	0	27	0

By Distance Band & # Clusters

	# Clusters			
Distance Band	**0**	**0, 1**	**1**	**1, 2**
within 1 mile	0	0	15,334	0
1–5 miles	0	0	22,182	0
5–10 miles	0	0	4,937	0
10–50 miles	0	0	2,050	0
> 50 miles	146	50	47	2

Businesses with no cluster generated had only two associated addresses; increasing the number of addresses generally resulted in clustering, though some remained unclustered regardless. A few businesses produced two separate clusters, but most had all addresses assigned to a single cluster regardless of the number of addresses available.

When examining clustering results by maximum distance detected, all entries with a coordinate difference of 50 miles or less were grouped into a single cluster. Heterogeneous clustering outcomes were only observed when coordinate differences exceeded 50 miles. These results demonstrate that `dbscan(eps = 1, minPts = 2)` both inconsistently associates nearby addresses and fails to distinguish larger coordinate differences, highlighting its limitations for this application.

99% of PO Boxes were successfully matched to a physical address. Of those that were not, either no physical address was listed for that business, or the distance between the PO Box and any available physical addresses was too large for them to be clustered together.

Algorithm Outcomes for Spatial Assignment Preparation

To minimize processing time, only addresses flagged as unlikely to have moved outside of their county were processed for dashboard visualization. These were further aggregated to their identified cluster to reduce the number of addresses processed in **Step 5**, a time-intensive step. Together, these measures ensured the data was ready for the front-end developer, allowing sufficient time for dashboard prototype development prior to the Summer 2025 symposium.

The threshold for selecting addresses unlikely to have moved between counties was arbitrarily set at 0.5 degrees longitude or latitude. Within this subset, 85% of businesses had a maximum relocation distance no larger than one block, and 2% had a relocation greater than 5 miles. No selected entry had a coordinate difference greater than 50 miles. Table 3 reflects the results seen in the previous section; entries with coordinate differences translating to large distances were clustered together.

Table 3: Row Count and Cluster Composition (by # Rows and by Distance Band)

By # Rows & # Clusters

	# Clusters	
# Rows	**0**	**1**
1	260,030	0
2	0	42,453
3	0	5,340
4	0	732
5	0	109
6	0	27

By Distance Band & # Clusters

	# Clusters	
Distance Band	**0**	**1**
within 1 block	260030	4,270
within 1 mile	0	15,334
1–5 miles	0	22,182
5–10 miles	0	4,937
10–50 miles	0	1,938
> 50 miles	0	0

Step 5: Point-in-Polygon Spatial Assignment

Development of this step and processing data through it were impeded by the short timeline leading up to the Summer 2025 symposium. Due to these time constraints, only 1% of the data could be evaluated. Of these, all were successfully associated with decennial boundaries for tract, county, and state across all three decennial periods, with no missing information.

Version: 006.07.15.26
Generated using Quarto (v. 1.8.27)
and RStudio (v. 2026.01.1+403)

Appendix

Optimizing Data Subsetting

For the implementation, refer to **SUBSECTION A1: Optimizing Data Subsetting** in "Code/Clean Raw Data_Step 1_2023 Format.R". The data processed in this step focused on entries for ABIs with suspected reduplicated addresses. Results were written to `"./Data/Results/From Clean Raw Data/Step 1_2023 Format/Step 1 MB Timings_Subsetting.csv"`.

Table 4: Codebook for Evaluation Output Fields

Field	Description
`expr`	Codename identifying the subsetting method applied.
`time`	Computational time elapsed to complete the operation, measured in milliseconds.

Optimizing Data Combination

For the implementation, refer to **SUBSECTION A2: Optimizing Data Combination** in "Code/Clean Raw Data_Step 1_2023 Format.R". The data processed in this step focused on entries for ABIs with suspected reduplicated addresses. Results were written to `"./Data/Results/From Clean Raw Data/Step 1_2023 Format/Step 1 MB Timings_Data Combining.csv"`.

Table 5: Codebook for Evaluation Output Fields

Field	Description
`expr`	Codename identifying the data combination method applied.
`time`	Computational time elapsed to complete the operation, measured in milliseconds.

Step 1: Address Deduplication

For the implementation, refer to **PART B: Standardize and Merge Similar Addresses** in "Code/Clean Raw Data_Step 1_2023 Format.R". The data processed in this step was the raw wide-format received in Summer 2025. The raw data was stored in `"./Data/Raw/KEEP LOCAL/church_wide_form_071723.csv"`, and the results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2023 Format/Step 1 Subsection B_04.22.2026.csv"`.

Table 6: Codebook for newly generated evaluation fields.
All other fields come from the imported raw dataset.

Field	Description
`compiled_address`	Formatted, combined version of all address elements.
`lonLat_test`	Boolean. TRUE if the difference between the maximum and minimum longitude and latitude values did not exceed the 0.002 degree threshold.

Step 1: Verify No Duplicates Got Added

For the implementation, refer to **SUBSECTION B1: Verify No Duplicates Got Added** in "Code/Clean Raw Data_Step 1_2023 Format.R". The data processed was the deduplicated version of the wide-format received in Summer 2025. The data processed was stored in `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2023 Format/Step 1 Subsection B_04.22.2026.csv"`, and the results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2023 Format/Step 1 Subsection B1_04.22.2026.csv"`.

Table 7: Codebook for Evaluation Output Fields

Field	Description
`abi`	Unique business identifier; evaluation is performed over each unique business ID.
`2001:2021`	Column-wise sum of all entries associated with the given business ID.
`all_counts_0_or_1`	Boolean. TRUE if all date entry sums for the given business ID are equal to 0 or 1.

Step 1: Override for Addresses With Same Physical Address Line 1

For the implementation, refer to **SUBSECTION B2: Explore the Nature of Duplications** in "Code/Clean Raw Data_Step 1_2023 Format.R". The data processed was a deduplicated subset of the wide-format data received in Summer 2025, restricted to the ABIs identified as containing erroneous duplicate year-opened and year-closed columns. The data processed was stored in `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2023 Format/Step 1 Subsection B_04.22.2026.csv"`, processed through **SUBSECTION B1**, and the results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2023 Format/Step 1 Subsection B2_04.26.2026.csv"`.

Table 8: Codebook for newly generated evaluation fields.
All other fields come from the results generated in **PART B**.

Field	Description
`override_duplicate`	Boolean. TRUE if the address was manually identified as the same physical address, indicating that the failed longitude and latitude similarity test should be overridden. FALSE if the failed test still applies. NA if this evaluation did not apply.

Step 1: Resolving Failed Geolocation Tests

For the implementation, refer to **PART C: Resolving Failed Geolocation Tests via Expansion** in "Code/Clean Raw Data_Step 1_2023 Format.R". The data processed were a

deduplicated subset of the wide-format data received in Summer 2025, restricted to the ABIs identified as containing erroneous duplicate year-opened and year-closed columns. The data processed was stored in `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2023 Format/Step 1 Subsection B2_04.26.2026.csv"`, and the results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2023 Format/Step 1 Subsection C1_04.27.2026.csv"`.

Table 9: Codebook for newly generated evaluation fields.
All other fields come from the results generated in **SUBSECTION B2**.

Field	Description
`same_num_clusters`	Expanded: addresses that were initially collapsed, failed the longitude and latitude similarity test, and required expansion for individual validation. TRUE: addresses that also failed the similarity test but clustered to an exact match and were kept collapsed. FALSE: other addresses associated with the same business where expansion assessment is not applicable.

Step 1: Final Result

For the implementation, refer to **PART D: Cleaning for Saving the Result** in "Code/Clean Raw Data_Step 1_2023 Format.R". The data processed was a deduplicated version of the wide-format received in Summer 2025, annotated with the columns generated in preceding sections. The `no_duplicates` data was stored in `"./Data/Raw/KEEP LOCAL/church_wide_form_071723.csv"` and excludes ABI in the `df_duplicates` dataset processed through **PART B** and **PART C**. The results were written to `Data/Results/KEEP LOCAL/From Clean Raw Data/Step 1_2023 Format/Step 01_Completed Result_04.29.2026.csv`.

No additional columns were generated after this evaluation.

Step 2: ABI-Specific Duplication Confirmation

For the implementation, refer to **SUBSECTION B1: ABI-Specific Duplication Confirmation** in "Code/Clean Raw Data_Step 2_2023 Format.R". The validated data was divided into three subsets based on how consolidated business address entries were handled after failing the geolocation similarity test and being expanded: those with no special cases

(`not_special_case`), those expanded due to a failed geocoordinate test and not sharing the same `address_line_1` (`fix_expanded`), and those marked for consolidation and geolocation override after expansion (`fix_diff_metadata`). These were all tested for presence of duplications in the years-open and years-closed information that would require reconciliation.

The processed data was stored in `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2023 Format/HPC Results"` and consolidated in **SUBSECTION A2: Compile the Results**. Results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2023 Format/Step 2 Subsection B_*"` as three files: `"..._All Other Data_06.04.2026.csv"`, `"..._Fix Expanded_06.04.2026.csv"`, and `"..._Fix Metadata_06.04.2026.csv"`.

Table 10: Codebook for the output fields produced by the evaluation.

Field	Description
`abi`	Unique business identifier. Evaluation is performed over each unique business ID.
`2001:2021`	Column-wise sum of all entries associated with the given business ID.
`all_counts_0_or_1`	Boolean. TRUE if all date entry sums for the given business ID are equal to 0 or 1.

Step 2: Final Result

For the implementation, refer to **PART C: Organize and Save the Results** in "Code/Clean Raw Data_Step 2_2023 Format.R". The data represents a recompilation of the subsets generated in **SUBSECTION B1: ABI-Specific Duplication Confirmation**, which were processed through subsequent sections only if flagged as requiring those steps. Businesses where duplication mitigation was unsuccessful were removed from this compilation prior to saving. Refer to **PART C: Organize and Save the Results**, `abi_sets` for details on how these datasets were recombined. The results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Step 2_2023 Format/Step 02_Completed Result_06.01.2026.csv"`.

Table 11: Codebook for newly generated evaluation fields.
All other fields come from the imported **Step 1** dataset.

Field	Description
`rowname`	Represents the row names of some processed subsets. Does not contain relevant information; this column can be ignored or removed.
`all address fields`	All address fields (`address_line_1`, `address_line_2`, `city`, `state`, `zipcode`, `zipcode_ext`, and `compiled_address`) reflect the validated address when `address_verified` is TRUE or `compiled_address` is not "No address match found"; otherwise, they reflect the original raw form. Note that `address_line_2` and `zipcode_ext` are fields new to this step.
`address_verified`	Boolean. TRUE if a verified address was retrieved from the USPS API database. FALSE if there was no match or an API retrieval error occurred.
`all_duplicates_corrected`	Boolean. From **SUBSECTION B2: Consolidate Duplications and Remove Difficult Cases**. TRUE if detected duplications were reconciled without adding or losing information. FALSE if the deduplication process introduced errors or could not reconcile the detected duplications. NA if the entry was not duplicated and this field is not applicable.

Step 3: Final Result

For the implementation, refer to **SUBSECTION A2: Save Results** in "Code/Clean Raw Data_Step 3_2023 Format.R". Data processing was limited by time constraints ahead of the Summer 2025 symposium. Approximately one-third of the **Step 2** data was run through the algorithm, imported from `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Summer 2025 Dashboard Prototype_ARCHIVED/Step 02_Church Wide_Convert to Preferred USPS Address_COMPLETED_06.07.2025"`. Results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Summer 2025 Dashboard Prototype_ARCHIVED/Step 03_Church Wide_Verified Geolocation_06.16.2025.csv.gz"`.

Table 12: Codebook for new output fields produced during the data cleaning and validation step. All other fields were present in the **Step 2** form of the data.

Field	Description
`verifiedLat`	The suggested latitude returned by matching the address. If multiple matches are returned by the geocoder, the first match is used.
`verifiedLon`	The suggested longitude returned by matching the address. If multiple matches are returned by the geocoder, the first match is used.
`similarLat`	Boolean. TRUE if the absolute difference between the given latitude and verified latitude is less than 1 degree; otherwise FALSE.
`similarLon`	Boolean. TRUE if the absolute difference between the given longitude and verified longitude is less than 1 degree; otherwise FALSE.
`verifiedGeo`	Stores the outcome of the geocoding attempt. TRUE indicates both coordinates were returned successfully; otherwise FALSE or a short message describing the issue (e.g., no match found, API request failed, or other warnings raised during geocoding).

Step 4: Final Result

For the implementation, refer to **SUBSECTION B3: Save Results** in "Code/Clean Raw Data_Step 4_2023 Format.R". Results from **Step 3** were subset to those with validated geolocations ($\approx$ 80% of ABI entries). Due to processing limitations from time constraints ahead of the Summer 2025 symposium, approximately half of this subset was run through the algorithm. Data was imported from `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Summer 2025 Dashboard Prototype_ARCHIVED/Step 03_Church Wide_Verified Geolocation_06.16.2025.csv.gz"`. Results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Summer 2025 Dashboard Prototype_ARCHIVED/Step 04_Cluster Addresses and Collapse By Area_06.17.2025.csv.gz"`.

Table 13: Codebook for new output fields produced during the data cleaning and validation step. All other fields were present in the **Step 3** form of the data.

Field	Description
`area`	The k-means cluster that the years-open and years-closed results represent.
`all address fields`	All address fields (`address_line_1`, `address_line_2`, `city`, `state`, `zipcode`, `zipcode_ext`, and `compiled_address`) selected from one of the addresses clustered to the same area and then aggregated together. Exact address details and geocoordinates do not correspond to the actual location for the years-open and years-closed information.
`other metadata`	All other metadata fields (`address_verified`, `lonLat_test`, and `verifiedGeo`) relate to the selected address associated with this cluster area.
`latitude/longitude`	The verified longitude/latitude of the selected address.
`2001:2021`	Column-wise sum of all entries associated with the given business ID and clustered area.
`all_counts_0_or_1`	Boolean. TRUE if all date entry sums for the given business ID are equal to 0 or 1.

Step 5: Final Result

For the implementation, refer to **SUBSECTION A1: tigris Algorithm** in "Code/Clean Raw Data_Step 5_2023 Format.R". All results generated in **Step 4** were passed through the algorithm; however, due to significant processing limitations imposed by time constraints ahead of the Summer 2025 symposium, only 1% of the data was successfully processed. Data was imported from `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Summer 2025 Dashboard Prototype_ARCHIVED/Step 04_Cluster Addresses and Collapse By Area_06.17.2025.csv.gz"`. Results were written to `"./Data/Results/KEEP LOCAL/From Clean Raw Data/Summer 2025 Dashboard Prototype_ARCHIVED/Step 05_Append Decennial Info Associated with an Area_05.27.2025.csv"`.

Table 14: Codebook for new output fields produced during the data cleaning and validation step. All other fields were present in the **Step 4** form of the data.

Field	Description
`decennial_census`	The census year the information represents: 2000, 2010, and 2020.
`tract`	The full tract-level GEOID, including the state and county Federal Information Processing Standards (FIPS) codes.
`tract_name`	The tract-specific information and name for that decennial year and geocoordinates.
`state_fips`	The state FIPS code for that decennial year and geocoordinates.
`county_fips`	The county FIPS code for that decennial year and geocoordinates.

References

1. Ushey, K., Wickham, H. & Posit. Introduction to renv.
2. Universal Service Administrative Co. (USAC). Geolocation methods.
3. Mark P.J. van der Loo. The stringdist package for approximate string matching. Rdocumentation. https://www.rdocumentation.org/packages/stringdist/versions/0.9.15/topics/stringdist (2014) doi:10.32614/RJ-2014-011.
4. Edit distance. Wikipedia (2026).
5. Srinivas Kulkarni. Jaro winkler vs levenshtein distance. Medium. https://srinivas-kulkarni.medium.com/jaro-winkler-vs-levenshtein-distance-2eab21832fd6 (2021).
6. Jaro–winkler distance. Wikipedia (2025).
7. Big o notation. Wikipedia (2026).
8. DSA time complexity. https://www.w3schools.com/dsa/dsa_timecomplexity_theory.php.
9. Adjacency list. Wikipedia (2026).
10. U.S. Geological Survey (USGS) & U.S. Department of the Interior. How much distance does a degree, minute, and second cover on your maps? https://www.usgs.gov/faqs/how-much-distance-does-a-degree-minute-and-second-cover-your-maps (2023).
11. National Oceanic and Atmospheric Administration (NOAA), National Hurricane Center & Central Pacific Hurricane Center. Latitude/longitude distance calculator. https://www.nhc.noaa.gov/gccalc.shtml.
12. City block. Wikipedia.

13. List of united states cities by area. Wikipedia.
14. U.S. Census Bureau. Census geocoder. https://geocoding.geo.census.gov/geocoder/.
15. United States Postal Service (USPS). Addresses 3.0 API. Developer portals. https://developers.usps.com/addressesv3.
16. harris-n-jalil-usps et al. Api-examples. United States Postal Service (2026).
17. Wickham, H. & Posit Software, PBC. Httr: Tools for working with URLs and HTTP. (2026).
18. Jeroen Ooms, Duncan Temple Lang & Lloyd Hilaiel. Jsonlite: A simple and robust JSON parser and generator for r. (2025).
19. SimpleMaps. United states cities database. https://simplemaps.com/data/us-cities.
20. U.S. Census Bureau. Geocoding services web application programming interface (API). (2026).
21. Hahsler, M., Piekenbrock, M., Arya, S., Mount, D. & Malzer, C. Dbscan: Density-based spatial clustering of applications with noise (DBSCAN) and related algorithms. (2026).
22. U.S. Census Bureau. TIGER/line shapefiles. Census.gov. https://www.census.gov/geographies/mapping-files/time-series/geo/tiger-line-file.html (2025).
23. Forrest, M. Spatial joins: A comprehensive guide - matt forrest. https://forrest.nyc/spatial-joins-a-comprehensive-guide/ (2025).
24. Kyle Walker & Matt Herman. Tidycensus: Load US census boundary and attribute data as 'tidyverse' and 'sf'-ready data frames. https://walker-data.com/tidycensus/index.html (2026).
25. Walker, K. & Rudis, B. Tigris: Load census TIGER/line shapefiles. (2025).
26. Edzer Pebesma et al. Simple features for r. https://r-spatial.github.io/sf/ (2018) doi:10.32614/RJ-2018-009.